Department of Electrical and Information Engineering

Faculty of Engineering, University of Ruhuna

Mini Project Report

EC8203 Applied Big Data Engineering

Hospital Patient Vital Signs Monitoring

A Kappa architecture for watching a ward in near real time

EG/2021/4417 **ASHFAQ M.R.M**

EG/2021/4644 **LAREEF J.M.**

EG/2021/4684 **MUNSIF M.F.A**

Computer Engineering

September 27, 2026

Source code: https://github.com/munsif-dev/hospital-vitals-kappa

Contents

1 The problem **2**

1.1 The two data sources . . . 2

1.2 Simulated time . . . 2

1.3 Three scripted patients . . . 2

2 What the system must do **2**

3 Architecture decision: Kappa, not Lambda **3**

3.1 The two options . . . 3

3.2 Why we chose Kappa . . . 3

3.3 What we give up . . . 3

3.4 When we would choose Lambda instead . . . 4

4 How the system is built **4**

4.1 Ingestion: Kafka . . . 5

4.2 Processing: one Spark stream . . . 5

4.3 Storage: Cassandra . . . 7

4.4 Serving and orchestration . . . 7

5 Why each technology **8**

6 Changing the scoring rule: the replay **8**

7 Observability **10**

8 Results **10**

9 Problems we found by checking against known answers **14**

10 Limitations and what we would do at production scale **15**

11 Conclusion **15**

A Individual contributions **16**

B How to run it **16**

C All screenshots **17**

1. The problem

A hospital ward has 40 beds. Each bed has a monitor that measures heart rate, oxygen level (SpO2), breathing rate, blood pressure and temperature. Once a day the pathology lab sends a file with blood test results for every patient. The ward wants to answer one question, taken from the assignment:

> **Business question**
> Which patients show concerning vital-sign trends right now, and how do yesterday's lab results change the risk picture for those patients going forward?

Nurses already use a standard score for this, the National Early Warning Score 2 (NEWS2) [1]. Each vital sign gets 0 to 3 points depending on how far it is from normal, and the points are added up. A total of 5 or more means "call a doctor soon"; 7 or more means "emergency". Our system calculates NEWS2 for every reading, adds points for abnormal blood tests, and shows the ward with the sickest patient at the top.

1.1. The two data sources

Both sources are Python programs that simulate the real thing.

- **Streaming source: bedside monitors.** One reading per bed every 15 simulated minutes, about 13.3 readings per real second for the whole ward. The numbers drift the way real vital signs do (each value wanders around the patient's own normal level, oxygen falling pushes the heart rate up, and so on). About 1.8 % of readings are deliberately broken, like a probe falling off, so we can check that the system catches them.
- **Daily source: the pathology lab.** One JSON file per simulated day at 06:00 with six tests per patient (lactate, white cells, CRP, creatinine, haemoglobin, potassium). One day's file is missing, one has a corrupted reference range, and some arrive late, so that every branch of the ingest workflow is exercised.

1.2. Simulated time

One simulated day lasts 5 real minutes, so the clock runs 288 times faster than real time. One real minute is 4.8 simulated hours. All containers read the same start time from one small file, so they agree on the simulated time. The run starts on 1 April 2026.

1.3. Three scripted patients

Most patients are stable. Three follow a script, so that we know in advance what the system should do with them:

- **P014** develops sepsis from day 2, 08:00. Temperature, heart rate and breathing rate climb and blood pressure falls. The day-2 blood tests agree (lactate 3.8, white cells 18.4). *The system must raise an alert.*
- **P031** has COPD, a lung disease. Their normal oxygen level is 89 to 91 %, which is low for most people but safe for them. *This patient is used to test the replay (Section 6).*
- **P007** has one heart-rate reading of 145 on day 1 at 14:00, then returns to normal. *The system must not raise an alert for one odd reading.*

2. What the system must do

Table 1 turns the use case into concrete requirements and shows where each one is met.

	Requirement	How it is met
R1	See a worsening patient within seconds, not at the end of the day	One stream scores every reading. Measured end-to-end time: 4.6 s median.
R2	Combine the daily lab file with the live vitals	Labs are published to a Kafka topic and joined to each reading as it is scored.
R3	Never lose a reading, and never score an impossible one	Impossible readings go to a dead-letter topic with their original bytes; late ones go to a late topic.
R4	One alert per problem, not one per reading	Alerts are grouped into episodes.
R5	Be able to change the scoring rule and re-score the past	Kafka keeps 30 simulated days; a second copy of the stream re-reads them with the new rule.
R6	A daily report and a live dashboard	Airflow renders a PDF each simulated day; Grafana and a REST API show the ward live.
R7	Notice when the system itself stops	Prometheus alert `WardMonitoringSilent`, and an Airflow health check.

Table 1: Requirements taken from the use case.

3. Architecture decision: Kappa, not Lambda

3.1. The two options

Lambda would have two paths. A speed layer would score readings as they arrive, and a batch layer would recompute everything from stored history each night. A serving layer would merge the two answers.

Kappa has one path. Every reading goes through one stream. History is kept in Kafka itself. To recompute the past, you start a second copy of the same stream at the beginning of the log.

3.2. Why we chose Kappa

1. The score must be the same everywhere. In Lambda the NEWS2 rule would exist twice, once in the stream code and once in the batch code. If they ever differ, even by one boundary (is a heart rate of 131 in the 2-point band or the 3-point band?), the ward screen and the daily report would disagree about the same patient. In a hospital that is not a small reporting error; it is a patient who looked fine on one screen and unwell on another. With Kappa the rule exists once, in `ward/clinical/news2.py`. A test reads the source code of the whole project and fails if any other file contains a scoring table.

2. The daily file is small reference data, not a big batch job. Each day's lab file has 240 rows. It does not need a separate batch engine; it needs to be joined to the stream. We publish it to a compacted Kafka topic (the newest result per patient and test is always kept), and the stream keeps it in memory for the join.

3. The main reason to recompute history is a rule change. Clinical rules change. NEWS2 itself changed in 2017 to add a second oxygen scale for COPD patients. When that happens we need to re-score past readings with the new rule and compare. In Kappa this is a replay of the log through the same code. In Lambda it is a batch job, which means writing the new rule twice.

4. Latency. Every reading must be scored within seconds. Both architectures can do that, so latency does not decide between them. But it rules out any design where the ward waits for a nightly job.

3.3. What we give up

Kappa is not free. We accept these costs:

- **Replay is slower than a bulk batch job** for very long histories, because it goes through the stream one micro-batch at a time. For 30 simulated days of one ward it is fast (Section 6); for

years of data from a whole hospital it would not be.

- **History lives in Kafka.** Kafka must keep 30 days of readings on disk. A data lake would be cheaper per gigabyte.
- **Compacted topics forget old values.** The labs topic keeps only the newest result per test. A replay therefore sees fewer historical lab results than the live stream did. We compare the NEWS2 part of the score, which does not use labs, and we say so wherever it matters.

3.4. When we would choose Lambda instead

We would switch if the daily source became a heavy analytic job (for example a nightly model trained on months of data from every ward), if history had to be kept for years, or if recomputing the past quickly mattered more than having one copy of the logic.

4. How the system is built

Figure 1 shows who uses the system. Figure 2 shows every service, the port it uses, and how data moves from top to bottom.

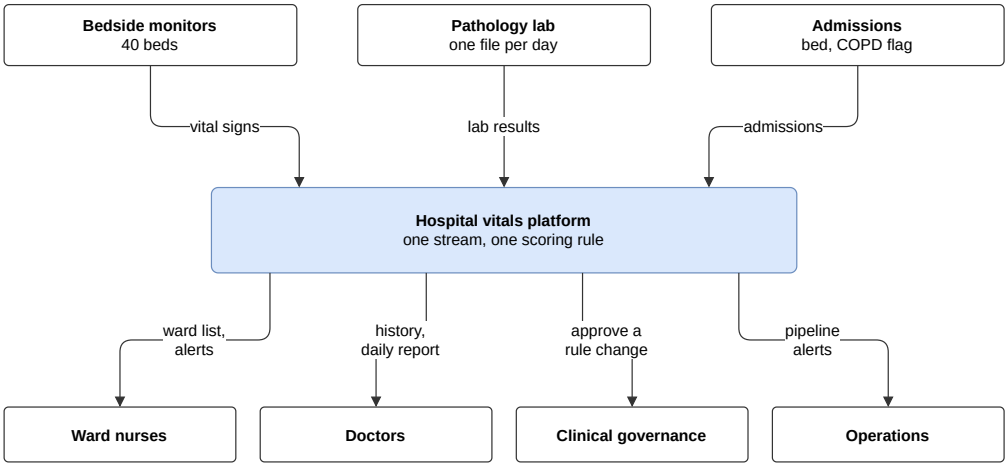


Figure 1: System context: two data sources in, four kinds of user out.

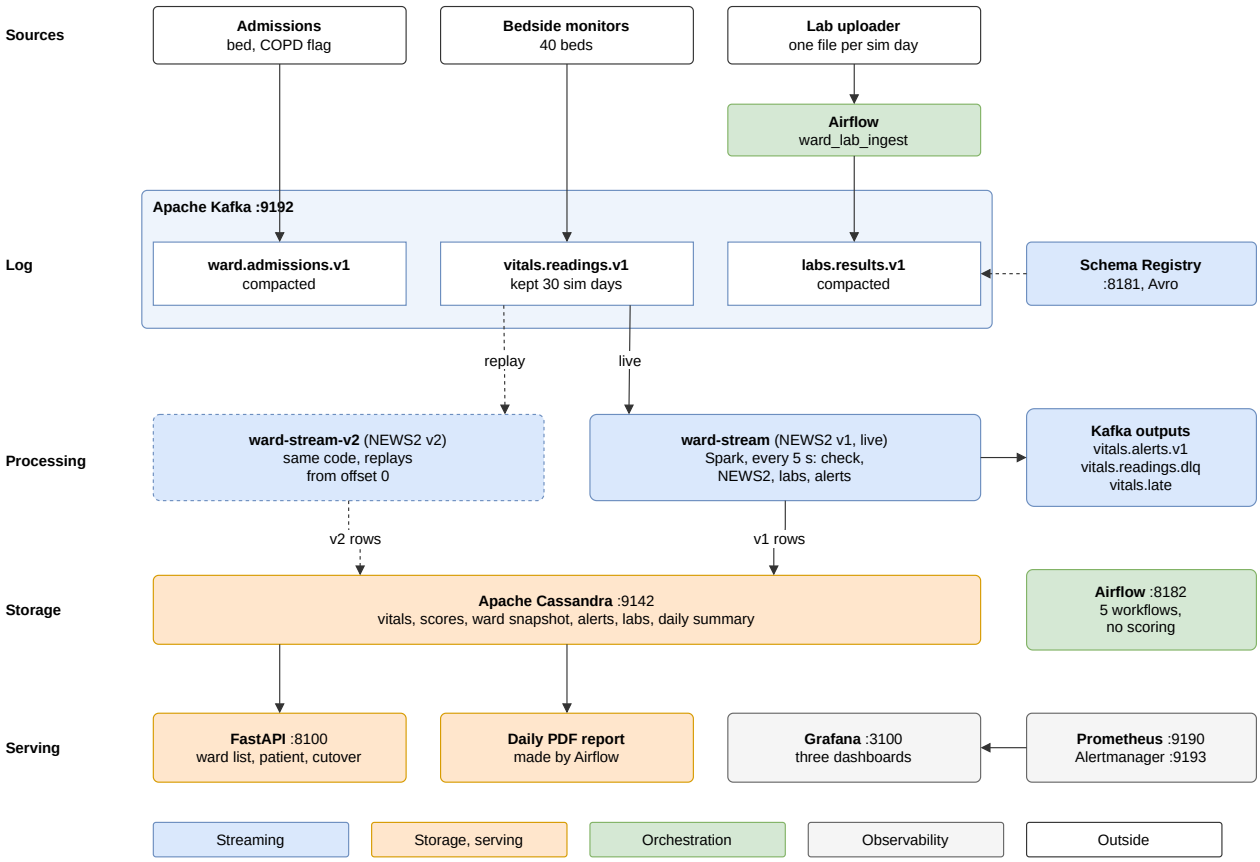


Figure 2: Layered architecture. Blue is streaming, orange is storage and serving, green is orchestration, grey is observability. Every service runs in Docker Compose.

4.1. Ingestion: Kafka

Kafka is the system of record. Table 2 lists the six topics. The cleanup policy matters: readings are deleted after 30 simulated days, while the admissions and labs topics are *compacted*, so the newest value for each key is kept forever. Every message is Avro, checked against a schema in the Schema Registry, and keyed by patient, so one patient's readings always arrive in order.

Topic	Partitions	Cleanup	What it holds
vitals.readings.v1	6	delete after 30 sim days	every bedside reading
labs.results.v1	3	compact	newest result per patient and test
ward.admissions.v1	3	compact	bed, age, condition, COPD flag
vitals.alerts.v1	3	delete	new alert episodes
vitals.readings.dlq	1	delete	impossible readings, with original bytes
vitals.late	1	delete	readings that arrived too late for their window

Table 2: Kafka topics (measured with `make topics`).

The lab file reaches Kafka through the Airflow workflow `ward_lab_ingest`. It checks the file's SHA-256 checksum, checks every row, moves a bad file to a quarantine folder, and otherwise publishes each row unchanged.

4.2. Processing: one Spark stream

The stream is a Spark Structured Streaming job. Every 5 real seconds it takes the new readings from Kafka and handles each one as shown in Figure 3.

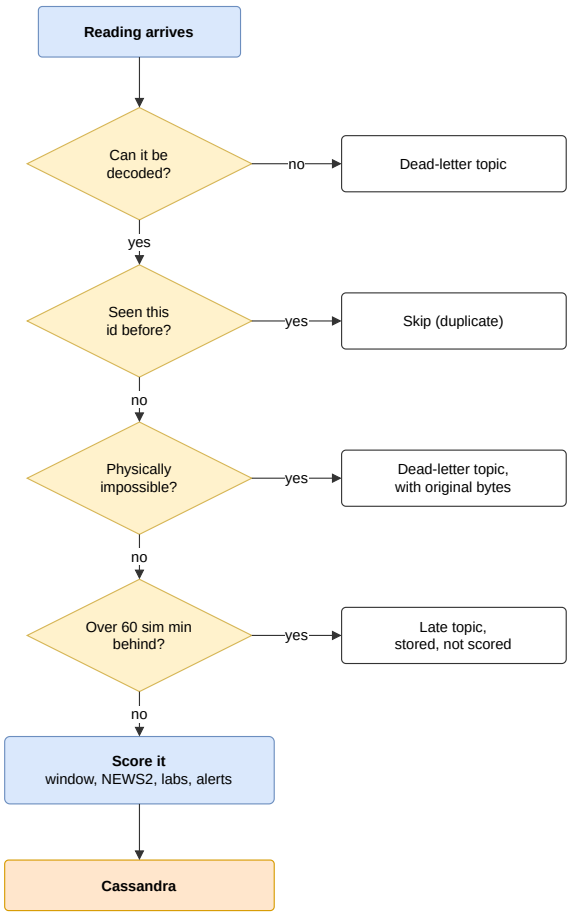


Figure 3: What happens to one reading.

The steps are:

1. **Check.** A reading that cannot be decoded, or that is physically impossible (SpO2 of 0, heart rate of 300, systolic pressure below diastolic), goes to the dead-letter topic with its original bytes and its exact position in Kafka. We use physical limits, not statistics. A statistical filter would throw away the most important readings, because a patient who is getting worse is, by definition, far from average.
2. **Duplicates and late readings.** A reading id seen before is skipped. A reading more than 60 simulated minutes older than the newest one for that patient goes to the late topic and is stored, but not scored.
3. **Trend.** The last 4 simulated hours (16 readings) are kept per patient, and a slope is fitted to each vital sign. We use the Theil-Sen slope (the median of all pairwise slopes), because one odd reading cannot move it much.
4. **Score.** NEWS2 is calculated by `news2.py`. Patients with the COPD flag use oxygen Scale 2 in version 2 of the rule.
5. **Labs.** Lab results reported *before* the reading add up to 5 points (for example +1 for lactate above 2.0). This gives the composite risk.
6. **Alerts.** Seven rules are checked, from "NEWS2 7 or more" to "two vital signs trending the wrong way". An alert is raised only when a new episode starts or its severity rises, and a single extreme reading must be confirmed by the next one.

Figure 4 follows patient P014 through these steps.

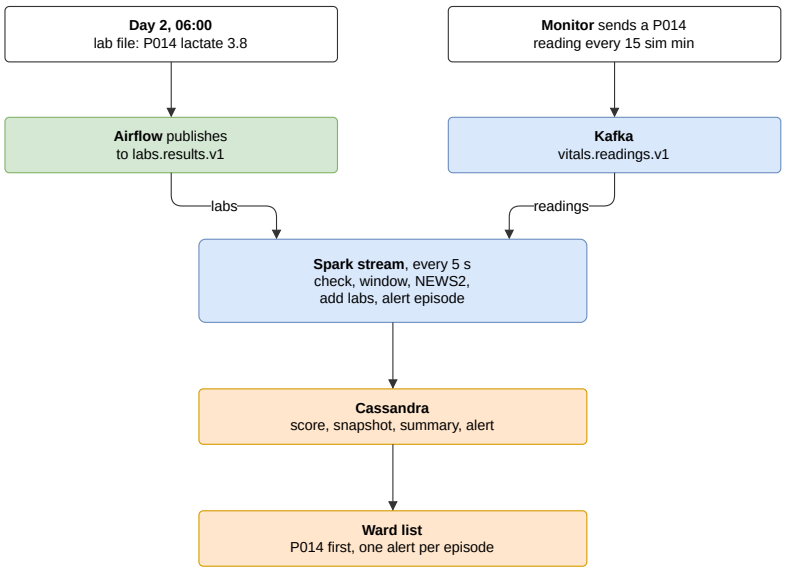


Figure 4: Event sequence for patient P014 on simulated day 2.

4.3. Storage: Cassandra

Scores land in Cassandra. We designed one table for each question the system has to answer, so every read goes to exactly one partition and no query needs `ALLOW FILTERING` (Figure 5). Three choices matter most:

- `ward_risk_snapshot` is ordered by risk score inside the partition, so the ward list comes back sickest-first without sorting.
- It has a 120-second time to live. If the stream stops, the ward list *empties* within two minutes. An empty screen is an obvious failure; a screen frozen on old values would look like a calm ward.
- Every score row carries its `scorer_version`, so version 1 and version 2 results sit side by side during a replay and nothing is overwritten.

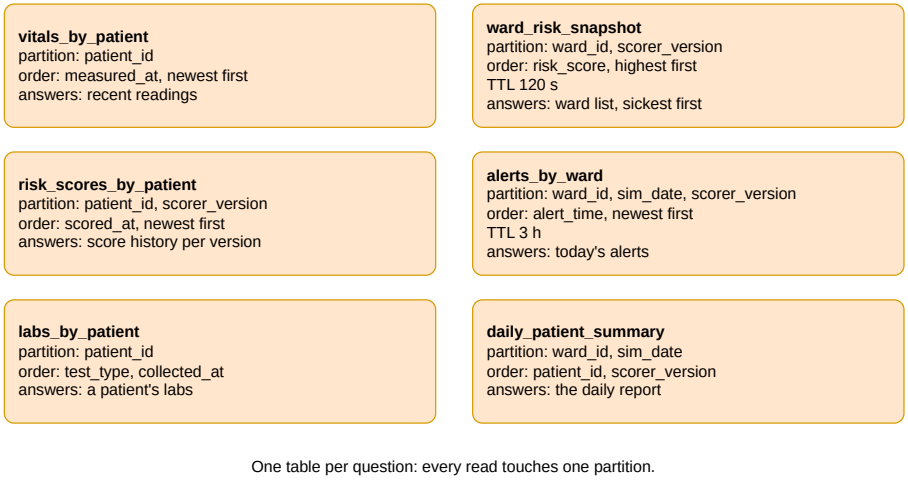


Figure 5: Cassandra tables and the query each one serves.

4.4. Serving and orchestration

A FastAPI service reads Cassandra and returns the ward list, a patient's history, labs, alerts, and the version comparison. Grafana shows three dashboards. Airflow runs five workflows (Table 3). Airflow only starts, checks and waits for things; it never calculates a score. A test fails if a workflow imports a data-processing library.

Workflow	Runs	What it does
ward_lab_ingest	every 5 min (1 sim day)	checksum, validate, quarantine or publish the lab file
ward_daily_report	every 5 min	wait for yesterday's summaries, render the PDF report
ward_healthcheck	every 2 min	fail if readings stop, the ward list is short, or the stream stalls
ward_retention	hourly	check the time-to-live settings and that the snapshot stays small
ward_replay	on demand	wait for the replay, compare versions, stop for human approval

Table 3: Airflow workflows.

5. Why each technology

Tool	Why it fits this use case	Alternative we considered
Apache Kafka (KRaft)	Keeps the history for replay; partitions by patient keep each patient's readings in order; compaction suits the labs and admissions.	RabbitMQ: deletes messages once read, so no replay.
Schema Registry, Avro	A reading that does not match the contract is rejected at the door, not deep inside the stream.	JSON without a schema.
Spark Structured Streaming	Required by the brief. Micro-batches every 5 s are far faster than the 15-minute reading interval.	Apache Flink: lower latency we do not need.
Apache Cassandra	Writes are cheap; the ward list is one partition read; time to live empties the screen if the stream stops.	PostgreSQL: fine at this size, but sorting and expiry would need extra work.
FastAPI	Typed responses and an automatic documentation page.	Flask.
Apache Airflow	Required by the brief. Good at "wait, check, branch, retry" for daily files.	cron: no retries, no history.
Prometheus, Alertmanager, Grafana	Pull-based metrics, alert rules as code, and routing of patient-safety alerts separately from engineering alerts.	Logs only.

Table 4: Technology choices.

6. Changing the scoring rule: the replay

This is the feature that justifies Kappa, so we tested it end to end. Version 1 of the rule scores every patient's oxygen on Scale 1. Version 2 uses Scale 2 for patients with the COPD flag. We wrote down the correct result before running it: *patients without COPD must not change at all.* For COPD patients, scores should fall when their usual oxygen level of 88 to 92 % is no longer counted as abnormal. Our first version of the check also said "and never rise". The first replay proved that wrong, and rightly so: Scale 2 also gives points to a COPD patient on oxygen whose level is 93 % or more, because too much oxygen is harmful for them. Every upward change we saw was exactly that case, so the check now allows it and fails on anything else. Figure 6 shows the steps.

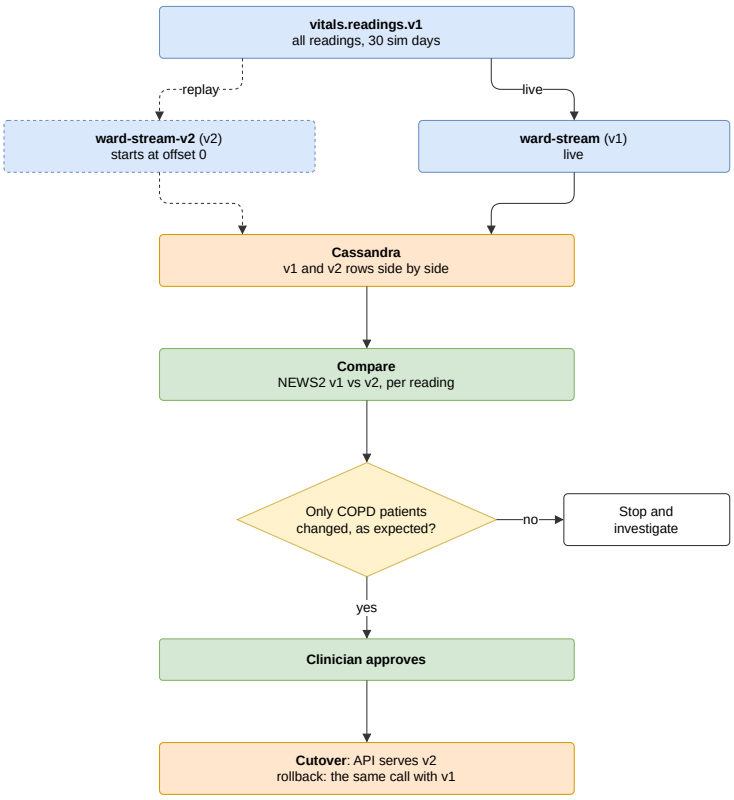


Figure 6: Replay and cutover.

`make replay` starts `ward-stream-v2`. It is the same image and the same code as the live stream, with three settings changed: the rule version, the starting position (the beginning of the log) and its own checkpoint. We measured:

Measurement	Result
Readings in the log at the start of the replay	15,800
Time for the v2 stream to catch up (real seconds)	126
Readings scored by both versions	17,822
Readings whose NEWS2 changed	4,001
of which COPD patients	4,001
of which other patients (must be 0)	0
Changes that went up (COPD, on oxygen at 93 % or more)	413
of which not explained by that (must be 0)	0
Readings that drop below NEWS2 5 under v2	1,035
P031: emergency (NEWS2 7 or more) alerts under v1 and v2	2 and 0

Table 5: Replay results, from `make diff`.

P031's emergency alerts disappear under v2, but its total number of alerts went up (27 under v1, 34 under v2). Under v1 its oxygen level kept it "red" all the time, which is one long alert episode. Under v2 its score sits near the thresholds and moves across them, and every crossing opens a new episode. A real system would add a margin before an episode closes; we list this as a limitation.

The Airflow workflow `ward_replay` runs the same comparison and ends at an approval step. Airflow does not switch versions itself. The switch is one API call (`make cutover VERSION=v2`); we tested it and the ward list changed from P031 at risk 8 to risk 5, with P014 unchanged at 10. Going back is the same call with "v1".

7. Observability

A monitoring system that fails silently is dangerous: an empty alert list looks exactly like a calm ward. So we measure the pipeline as carefully as the patients.

What	How	Why
Structured logs	JSON lines from every service, each with `service` and `stage` (ingest, process, store, serve, orchestrate).	One query finds every log line for one stage across all containers.
Readings in, scores out	Counters `readings_produced_total`, `risk_scores_written_total`.	If scores stop while readings continue, the stream is dead.
End-to-end time	Histogram from the monitor's timestamp to the Cassandra write.	This is what a nurse waits for.
Micro-batch progress	A Spark query listener records batch time, rows and last progress.	Tells a stalled query from an idle one.
Injected vs dead-lettered	Counters by fault type on both sides.	The injector is a control: what goes in must come out.
Replay progress	Per-partition offsets of the replay stream.	Spark does not use consumer groups, so consumer lag cannot measure it.

Table 6: What is measured, how, and why.

Seven Prometheus rules watch these signals. The most important is `WardMonitoringSilent`: no scores written for one minute. It is written so that it still fires when the stream container has disappeared completely, not only when it is idle. Alertmanager sends it to two receivers, the ward and the on-call engineer, because it is both a patient-safety and a technical problem.

We tested it by stopping the stream (`make chaos-stop-stream`). The alert moved to pending after 57 s and fired after 114 s, and it resolved 37 s after the stream was restarted (half of that is Spark starting). No reading was lost: the stream restarted from its checkpoint and worked through the backlog, and the day's summary carried on from what was stored (99 readings per patient for that day, the same as the replay stream counted). A second rule, `StaleLabDataElevated`, fired on its own later in the run, when the simulated lab withheld the day-5 file and the day-4 file was quarantined, so no patient had lab results newer than two days. A test checks that every metric named in an alert rule or a dashboard is really produced by the code; the first version of the project had an alert on a metric that did not exist, so it could never fire.

We did not add distributed tracing. Each dead letter carries the exact Kafka partition and offset of its reading, which is enough to find it, but a trace across all services would be the next step.

8. Results

We ran the whole stack from nothing (`make clean`, `make up`) and measured it. Table 7 gives the numbers; the screenshots below and in Appendix C are from that run.

Measurement	Result
Readings delivered to Kafka per second (expected about 12.8)	13.3
Distinct patient keys on the vitals topic (expected 40)	40
Impossible readings injected (snapshot)	406
Of those, found in the dead-letter topic (Table 8)	406
End-to-end time, median and 95th percentile (real seconds)	4.6 / 7.3
Micro-batch duration, typical (real seconds)	2.5
P014: first NEWS2 7+ alert after sepsis onset (simulated time)	3 h 3 min
P014: NEWS2 7+ alerts raised in that episode (expected 1)	1
P007: alerts within 2 simulated hours of the spike (expected 0)	0
API ward list response time, median	13 ms
Memory used by the whole stack	5.9 GB
Automated tests passing	533

Table 7: Measured results.

Table 8 checks the dead-letter path against the fault injector, reason by reason. We read the injector's counter, waited 20 seconds, and counted the dead-letter topic by distinct Kafka position. Every injected fault was there under its own reason, and no other reason appeared, so no real reading was rejected. (In an earlier run this check found 16 extra "inverted blood pressure" rejections: real P014 readings, made impossible by a bug in the sepsis script. That bug was fixed before this run.)

Reason	Injected	In the dead-letter topic 20 s later
SpO2 probe detached	124	128
Heart rate out of range	86	87
Empty reading	75	78
Blood pressure inverted	51	51
Temperature out of range	50	51
Timestamp in the future	20	20
Total	406	415

Table 8: Injected faults against dead letters. The right column is slightly higher because the monitors kept injecting during the 20 seconds. Nine duplicate readings were also injected; they are skipped by id, not dead-lettered.

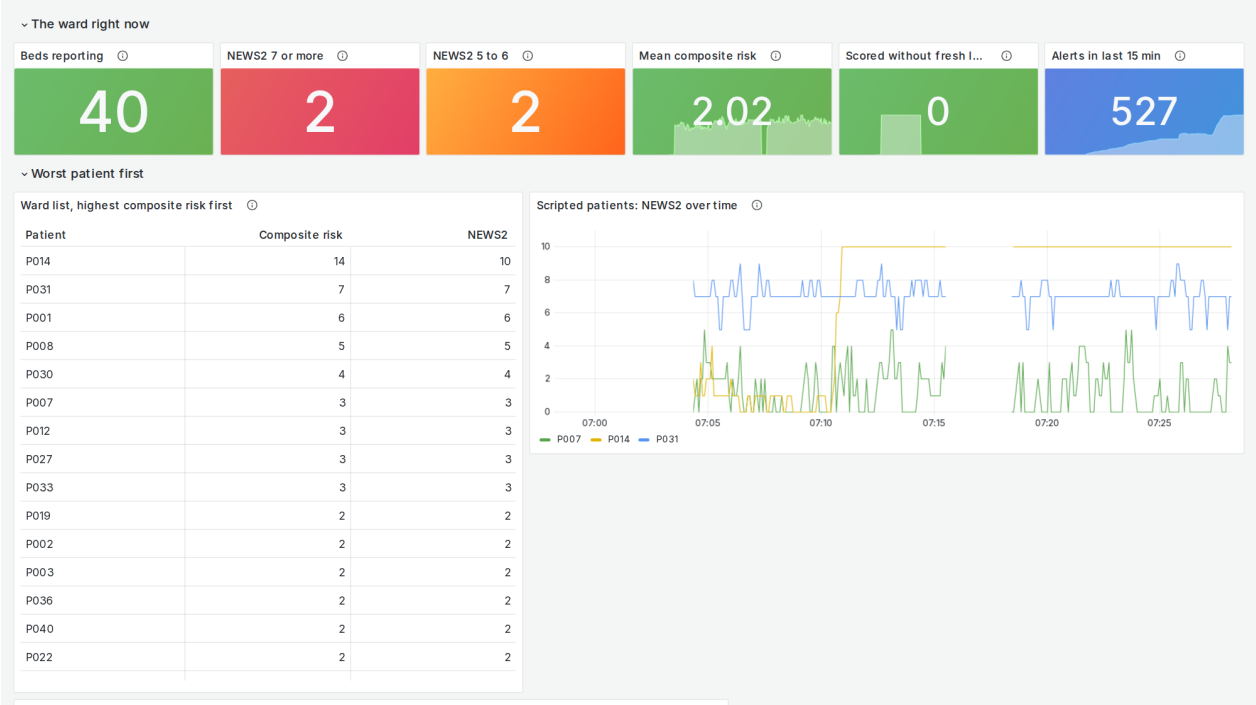
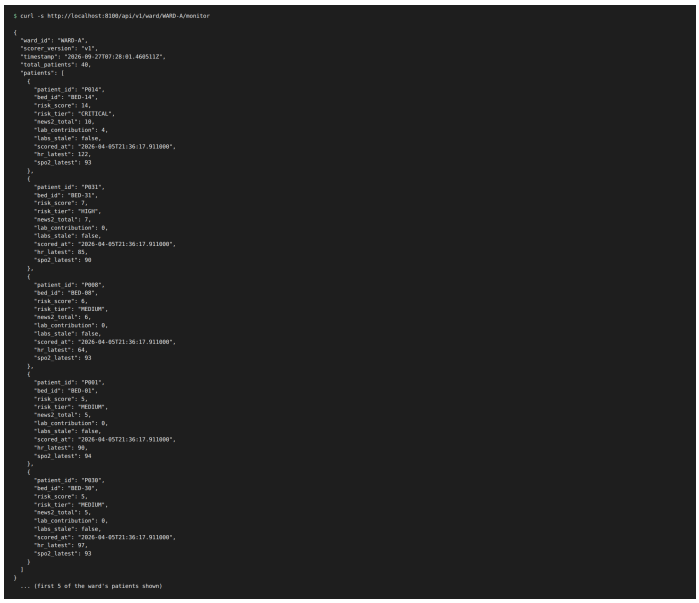


Figure 7: Grafana ward monitor during the run. The table lists all 40 beds with the highest composite risk first. The chart shows the three scripted patients: P014 rising on day 2, P031 steady, P007 with one spike.



```
$ curl -s http://localhost:8100/api/v1/ward/WARD-A/monitor

{
  "ward_id": "WARD-A",
  "scorer_version": "v1",
  "timestamp": "2026-09-27T07:28:01.460511Z",
  "total_patients": 40,
  "patients": [
    {
      "patient_id": "P014",
      "bed_id": "BED-14",
      "risk_score": 14,
      "risk_tier": "CRITICAL",
      "news2_total": 10,
      "lab_contribution": 4,
      "labs_stale": false,
      "scored_at": "2026-04-05T21:36:17.911000",
      "hr_latest": 122,
      "spo2_latest": 93
    },
    {
      "patient_id": "P031",
      "bed_id": "BED-31",
      "risk_score": 7,
      "risk_tier": "HIGH",
      "news2_total": 7,
      "lab_contribution": 0,
      "labs_stale": false,
      "scored_at": "2026-04-05T21:36:17.911000",
      "hr_latest": 85,
      "spo2_latest": 98
    },
    {
      "patient_id": "P008",
      "bed_id": "BED-08",
      "risk_score": 6,
      "risk_tier": "MEDIUM",
      "news2_total": 6,
      "lab_contribution": 0,
      "labs_stale": false,
      "scored_at": "2026-04-05T21:36:17.911000",
      "hr_latest": 64,
      "spo2_latest": 93
    },
    {
      "patient_id": "P001",
      "bed_id": "BED-01",
      "risk_score": 5,
      "risk_tier": "MEDIUM",
      "news2_total": 5,
      "lab_contribution": 0,
      "labs_stale": false,
      "scored_at": "2026-04-05T21:36:17.911000",
      "hr_latest": 99,
      "spo2_latest": 94
    },
    {
      "patient_id": "P030",
      "bed_id": "BED-30",
      "risk_score": 5,
      "risk_tier": "MEDIUM",
      "news2_total": 5,
      "lab_contribution": 0,
      "labs_stale": false,
      "scored_at": "2026-04-05T21:36:17.911000",
      "hr_latest": 97,
      "spo2_latest": 93
    }
  ]
}
  ... (first 5 of the ward's patients shown)
```

Figure 8: The same ward list from the REST API (`/api/v1/ward/WARD-A/monitor`), sickest first.

```
cqlsh> SELECT alert_time, patient_id, alert_type, severity, news2_total, composite_risk FROM ward.alerts_by_ward WHERE ward_id='WARD-A' AND sim_date='2026-04-02' AND scorer_version='v1' AND alert_time >

 alert_time                      | patient_id | alert_type          | severity | news2_total | composite_risk
---------------------------------+------------+---------------------+----------+-------------+----------------
 2026-04-02 11:03:48.213000+0000 |       P014 |          NEWS2_HIGH | CRITICAL |           7 |              7
 2026-04-02 11:03:48.213000+0000 |       P019 | RAPID_DETERIORATION | CRITICAL |           6 |              6
 2026-04-02 11:03:48.213000+0000 |       P018 |    TREND_CONCERNING |   MEDIUM |           3 |              3
 2026-04-02 10:48:48.227000+0000 |       P019 |        NEWS2_MEDIUM |     HIGH |           6 |              6
 2026-04-02 10:40:52.330000+0000 |       P018 | RAPID_DETERIORATION | CRITICAL |           5 |              5
 2026-04-02 10:40:52.330000+0000 |       P014 | RAPID_DETERIORATION | CRITICAL |           6 |              6
 2026-04-02 10:25:52.314000+0000 |       P029 |    SINGLE_PARAM_RED |     HIGH |           3 |              3
 2026-04-02 10:25:52.314000+0000 |       P001 |    SINGLE_PARAM_RED |     HIGH |           6 |              6
 2026-04-02 10:25:52.314000+0000 |       P014 |        NEWS2_MEDIUM |     HIGH |           6 |              6
 2026-04-02 10:10:52.351000+0000 |       P008 | RAPID_DETERIORATION | CRITICAL |           7 |              7
 2026-04-02 09:55:52.316000+0000 |       P014 |    TREND_CONCERNING |   MEDIUM |           4 |              4
 2026-04-02 09:55:52.316000+0000 |       P018 |        NEWS2_MEDIUM |     HIGH |           5 |              5
 2026-04-02 09:40:52.344000+0000 |       P008 |          NEWS2_HIGH | CRITICAL |           7 |              7

(13 rows)
```

Figure 9: Real `cqlsh` output: today's alerts. Each condition appears once per episode, not once per reading.

Ward WARD-A daily risk report

Consolidated 24-Hour Patient Risk & Pathology Correlation Report

Ward: WARD-A (Acute Medical Unit)
Simulated Date: 2026-04-02
Generated At: 2026-09-27 08:06:19 UTC
Scorer Version: v1

MONITORED BEDS	CRITICAL / DETERIORATING	ELEVATED RISK (HIGH/ MED)	TOTAL ALERTS EMITTED
40	2	5	95
100% telemetry coverage	24 rapid trajectories	1 High · 4 Medium	0 patients with stale labs

Clinical Deterioration & Risk Escalation Events

Bed BED-30, patient P030 (copd) RAPID DETERIORATION
Bed BED-30 (copd) reached NEWS2 11 and composite risk 11; the day ended at HIGH. 11 alert episode(s), highest severity CRITICAL. NEWS2 of 7 or more calls for an emergency clinical response.

Bed BED-14, patient P014 (sepsis_risk) RAPID DETERIORATION
Bed BED-14 (sepsis_risk) reached NEWS2 10 and composite risk 10; the day ended at CRITICAL. 4 alert episode(s), highest severity CRITICAL. NEWS2 of 7 or more calls for an emergency clinical response.

Bed BED-01, patient P001 (copd) RAPID DETERIORATION
Bed BED-01 (copd) reached NEWS2 9 and composite risk 9; the day ended at MEDIUM. 8 alert episode(s), highest severity CRITICAL. NEWS2 of 7 or more calls for an emergency clinical response.

Bed BED-31, patient P031 (copd) RAPID DETERIORATION
Bed BED-31 (copd) reached NEWS2 9 and composite risk 9; the day ended at MEDIUM. 3 alert episode(s), highest severity HIGH. NEWS2 of 7 or more calls for an emergency clinical response.

Bed BED-08, patient P008 (copd) RAPID DETERIORATION
Bed BED-08 (copd) reached NEWS2 8 and composite risk 8; the day ended at MEDIUM. 5 alert episode(s), highest severity CRITICAL. NEWS2 of 7 or more calls for an emergency clinical response.

Bed BED-02, patient P002 (copd) RAPID DETERIORATION
Bed BED-02 (copd) reached NEWS2 7 and composite risk 7; the day ended at MEDIUM. 7 alert episode(s), highest severity CRITICAL. NEWS2 of 7 or more calls for an emergency clinical response.

Bed BED-27, patient P027 (cardiac) RAPID DETERIORATION
Bed BED-27 (cardiac) reached NEWS2 7 and composite risk 7; the day ended at LOW. 5 alert episode(s), highest severity CRITICAL. NEWS2 of 7 or more calls for an emergency clinical response.

Bed BED-03, patient P003 (copd) RAPID DETERIORATION
Bed BED-03 (copd) reached NEWS2 6 and composite risk 6; the day ended at LOW. 6 alert episode(s), highest severity CRITICAL. NEWS2 of 5 or more calls for an urgent clinical review.

Bed BED-10, patient P010 (sepsis_risk) RAPID DETERIORATION
Bed BED-10 (sepsis_risk) reached NEWS2 6 and composite risk 6; the day ended at LOW. 2 alert episode(s), highest severity HIGH. NEWS2 of 5 or more calls for an urgent clinical review.

Bed BED-12, patient P012 (copd) RAPID DETERIORATION

Simulated data - Hospital Vitals Kappa coursework

Page 1 of 4

Figure 10: First page of the daily PDF report produced by Airflow from the stored daily summaries.

Figure 11: Replay dashboard after the v2 stream caught up. P031 (COPD) scores lower under v2; P014 is identical under both.

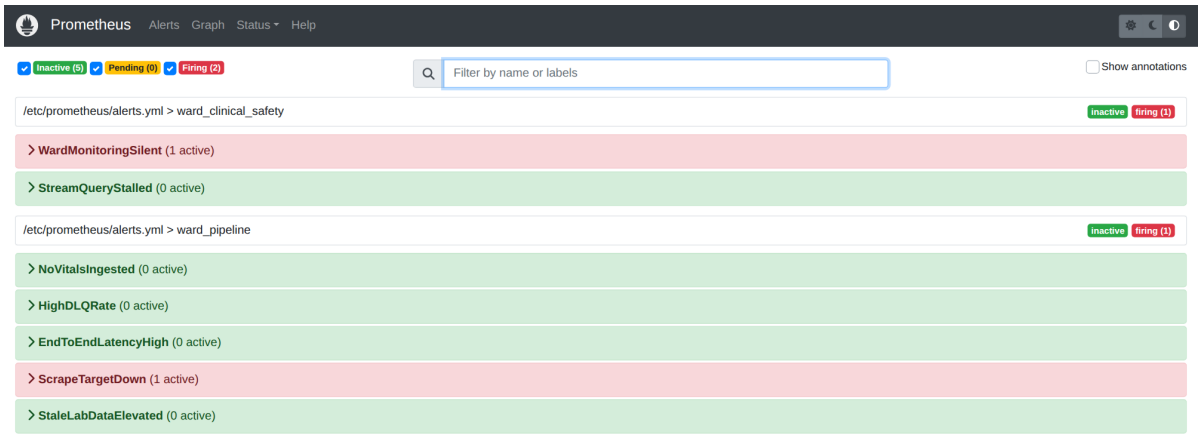


Figure 12: Prometheus during the outage test: `WardMonitoringSilent` firing after the stream was stopped.

9. Problems we found by checking against known answers

Many faults in this project raised no error at all. They were found by comparing an output with a value we knew in advance: 40 patient keys, the injector's own fault counts, zero changes for non-COPD patients, one alert for P014. The most instructive ones:

- No process could connect to Cassandra: the connection code passed an argument the driver does not accept. Every unit test replaced the database with a fake, so it only showed up on the first real run.

- The same alert was raised again every 15 simulated minutes, because its id was built from a time that moved with every reading. Alerts are now grouped in episodes.
- After the outage test, readings stamped two hours in the future were accepted, because the check compared them with the clock at processing time, which was far ahead while the stream caught up. It now compares with the time the reading was sent.
- P007's single spike was written into the patient's state, so the next two readings stayed high; later, a small clock jump put two readings in its time window. The monitor now sends it exactly once.
- Lab results were stored as "LACTATE" but looked up as "lactate", so labs never added any points. The live join also only read the labs topic once, at start-up.
- The dead-letter rate was almost double the injected rate, because oxygen values could drift above 100 % and were rejected as impossible.
- The API started "healthy" without a database connection, because an error on start-up was caught and ignored.
- After a restart in the middle of a day, the stream overwrote that day's summary with the few readings it had seen since, and joined the backlog with the wrong day's labs.
- The injector check found 16 real P014 readings rejected as impossible: the sepsis script lowered systolic pressure but not diastolic.
- The alert on the most important condition, a silent stream, named a metric that nothing produced, and most dashboard panels fell back to made-up constants.
- Most services printed plain-text logs although JSON was configured, because their loggers were created before the configuration was applied.

Each fix came with a test that failed before it. Where a test guards against a whole class of mistakes (only one scoring rule, every alert names a real metric), we planted a mistake to prove that the test catches it.

10. Limitations and what we would do at production scale

- **One machine.** Kafka has one broker and Cassandra one node, with no replication. In production each would have at least three.
- **Scoring is done record by record in Python.** Spark reads the micro-batch, but the rows are collected to the driver and scored one at a time. This is fine for 13 readings per second; for a whole hospital the scoring would have to run inside Spark's workers.
- **Stream state is in memory.** The daily totals are reloaded from Cassandra after a restart, but the 4-hour windows and alert episodes start again. After the outage test, conditions that were already open were raised again: P014 has 7 alert episodes under v1 and 5 under v2 over the same days, with identical scores. Spark's own state store would keep them.
- **Replay sees fewer labs.** Because the labs topic is compacted, see Section 3.
- **Simulated data and untuned alerts.** The physiology is realistic enough to test the pipeline, but the alert rules have not been checked against real patients. On our simulated ward they still fire for some stable patients (about 600 alert episodes over six simulated days). An episode also closes as soon as a score dips below its threshold, so a patient near a threshold gets repeated alerts (P031 under v2). A margin before closing an episode, agreed with clinicians, would fix both.
- **No tracing and no security.** There is no login, no encryption and no audit trail, all of which patient data would need.

11. Conclusion

We built a Kappa pipeline for a hospital ward. One Spark stream scores every reading with one copy of the NEWS2 rule, joins the daily lab results, and stores the result in Cassandra for a live

ward list, an API and a daily report. Kafka keeps the history, and changing the scoring rule is a replay of that history through the same code. We checked every result against an answer we knew in advance, and the system met them: P014 is caught, P007 is not flagged, and only COPD patients change under the new rule.

References

[1] Royal College of Physicians. *National Early Warning Score (NEWS) 2: Standardising the assessment of acute-illness severity in the NHS.* London, 2017.

[2] J. Kreps. "Questioning the Lambda Architecture." O'Reilly Radar, 2014.

[3] M. Kleppmann. *Designing Data-Intensive Applications.* O'Reilly, 2017.

[4] Apache Software Foundation. *Structured Streaming Programming Guide*, Spark 3.5.

A. Individual contributions

All three members took part in design discussions, testing and reviewing the report. The major parts each person built are:

Member	Major parts
Munsif	The NEWS2 scoring rule and its tests; the physiology simulator and the three scripted patients; the Spark stream (checking, windows, scoring, lab join, alert episodes, dead-letter and late paths); the metrics and Prometheus alert rules; writing this report.
Ashfaq	The Docker Compose platform (Kafka, Schema Registry, topics); the Cassandra data model and data access code; the Airflow workflows; the FastAPI service; the replay stream and the version comparison; the Grafana dashboards; the architecture diagrams.
Lareef	The architecture decision records; the daily PDF report; the test suites for the API, the replay and the observability configuration; the screenshots and the measurements used in this report.

B. How to run it

```
make clean && make up     start everything from nothing
make topics               list topics with their cleanup policies
make keys                 count patient keys on the vitals topic (40)
make replay               start the v2 stream from offset 0 and wait
make diff                 compare v1 and v2
make cutover VERSION=v2   switch the version the API serves
make chaos-stop-stream    stop the stream to test the alert
make test                 run the automated tests
```

Ports: Kafka UI 8180, API 8100, Airflow 8182 (admin, admin), Grafana 3100, Prometheus 9190, Alertmanager 9193, Spark UI 4140 (replay 4141), Cassandra 9142.

C. All screenshots

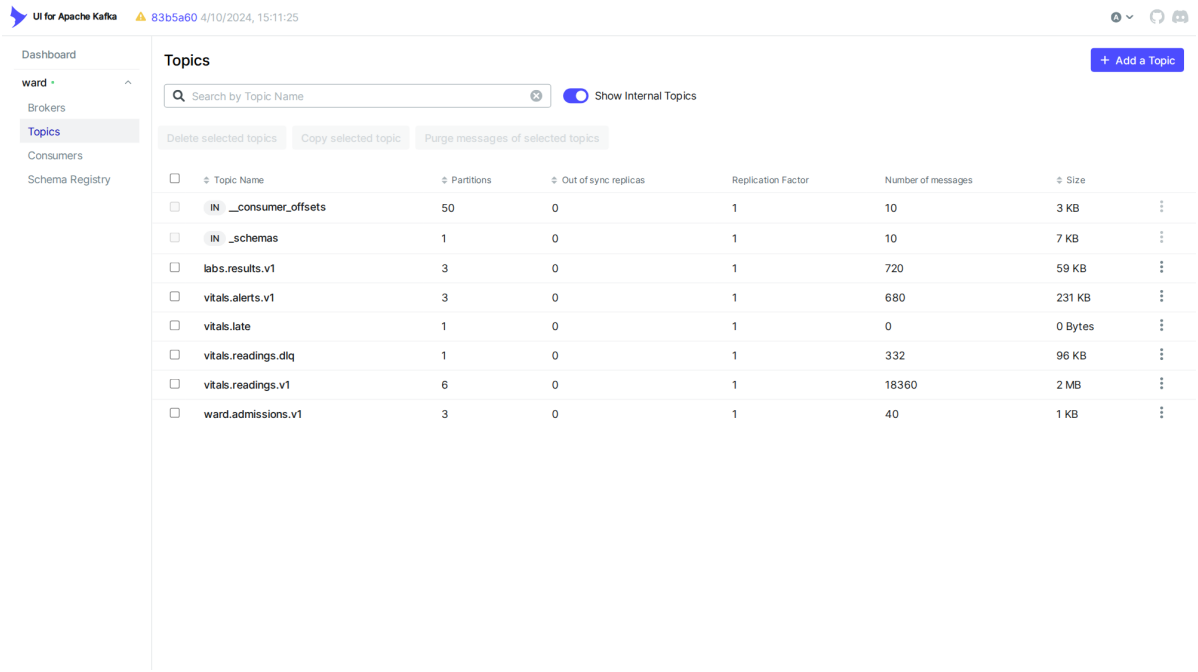


Figure 13: Kafka UI: the six topics.

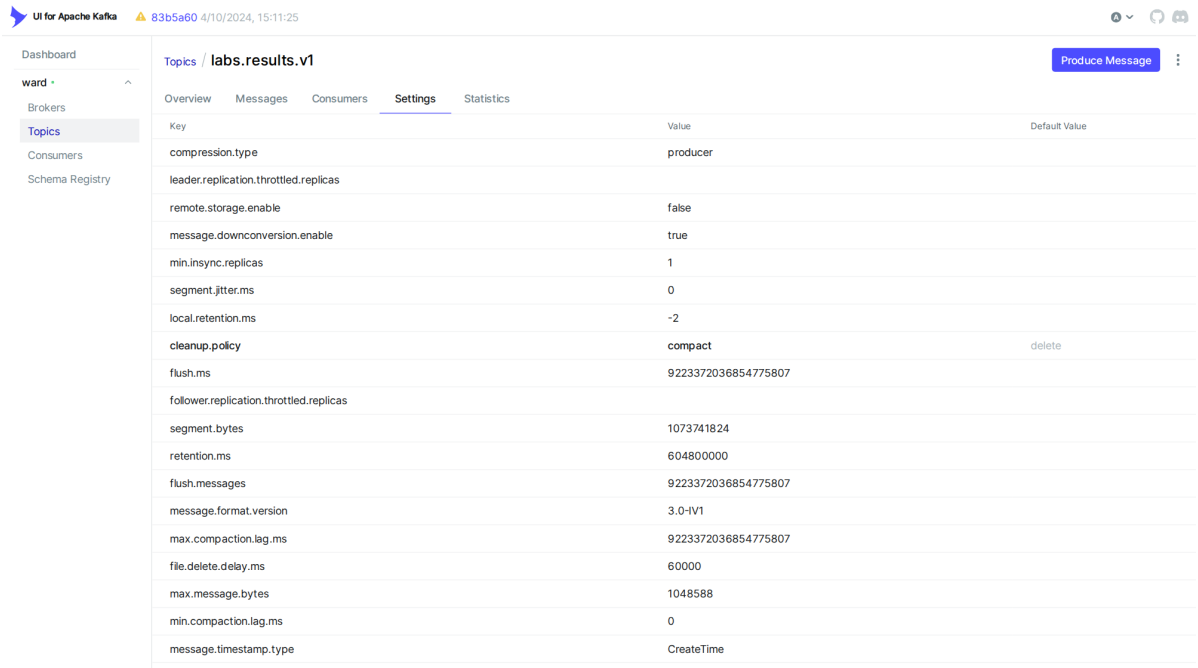


Figure 14: Kafka UI: `labs.results.v1` is compacted.

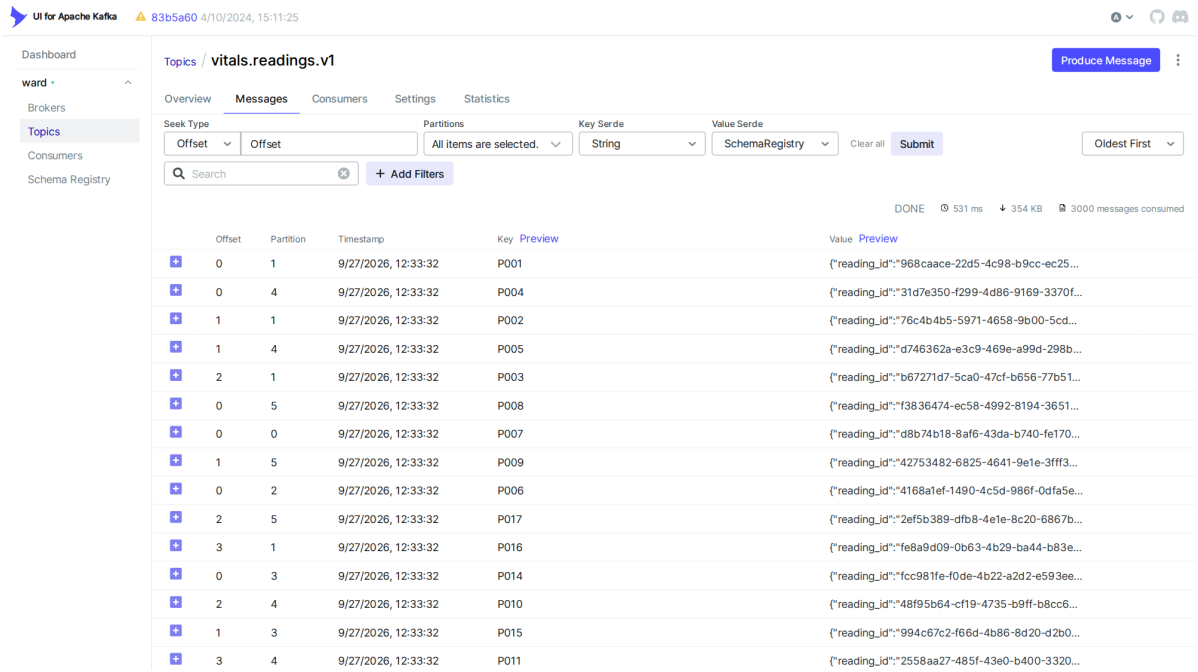


Figure 15: Kafka UI: readings on `vitals.readings.v1`, decoded from Avro through the Schema Registry, keyed by patient.

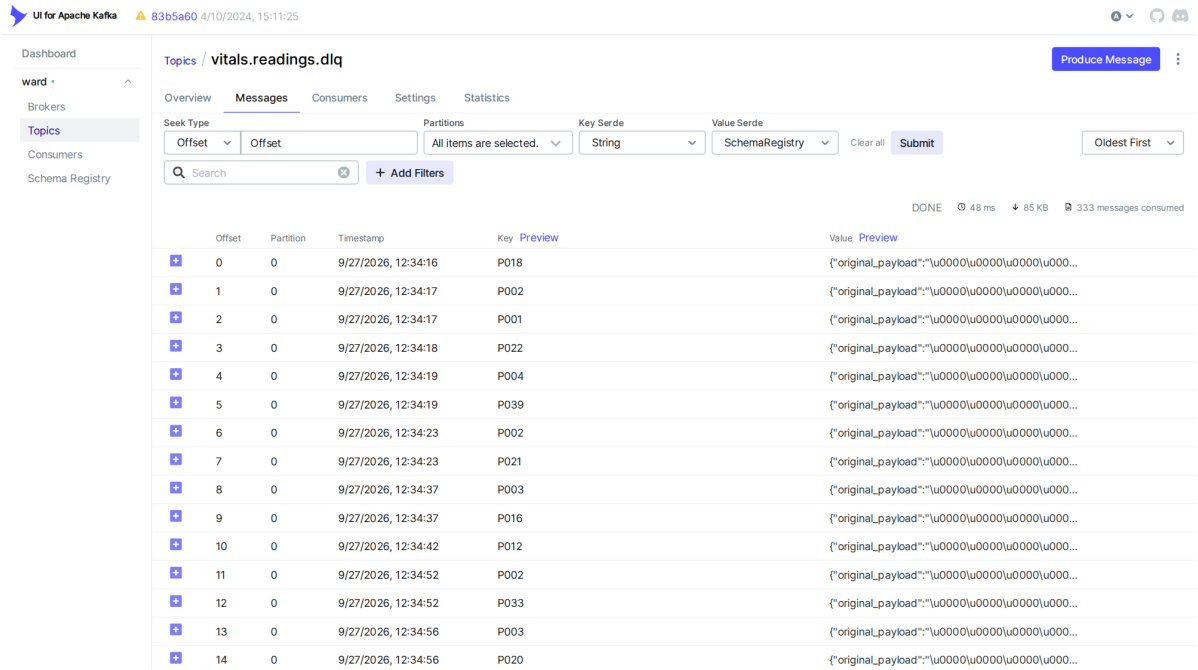


Figure 16: Kafka UI: dead-lettered readings with their rejection reason.

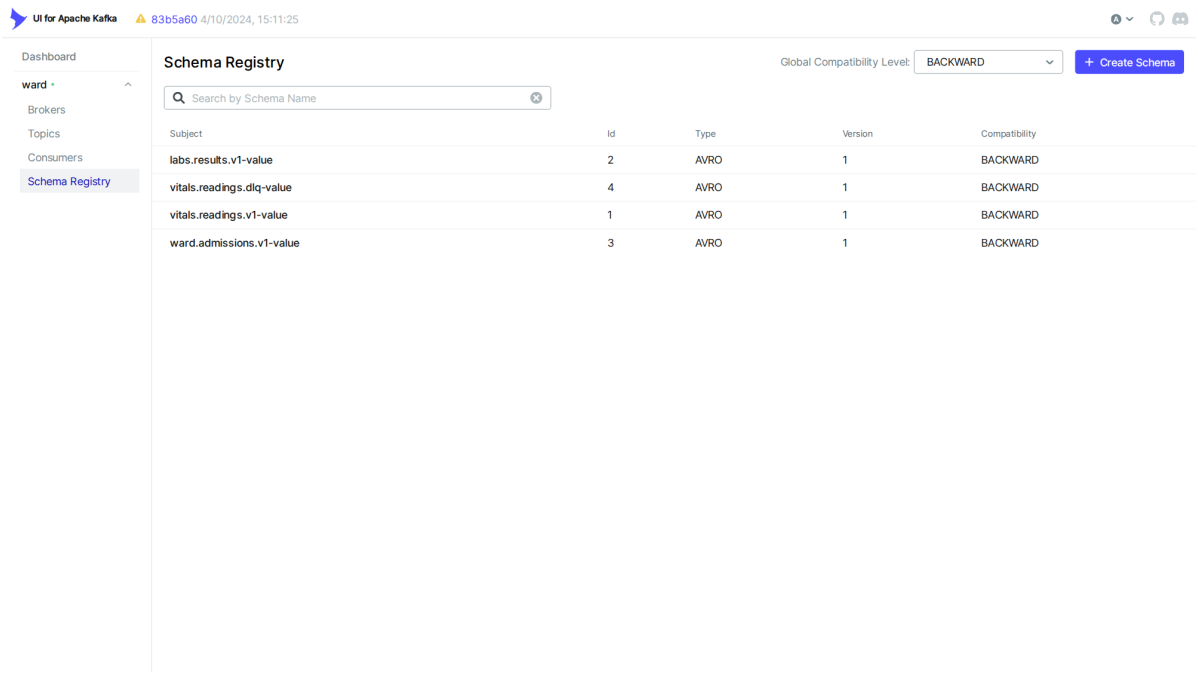


Figure 17: Kafka UI: the four Avro schemas in the Schema Registry.

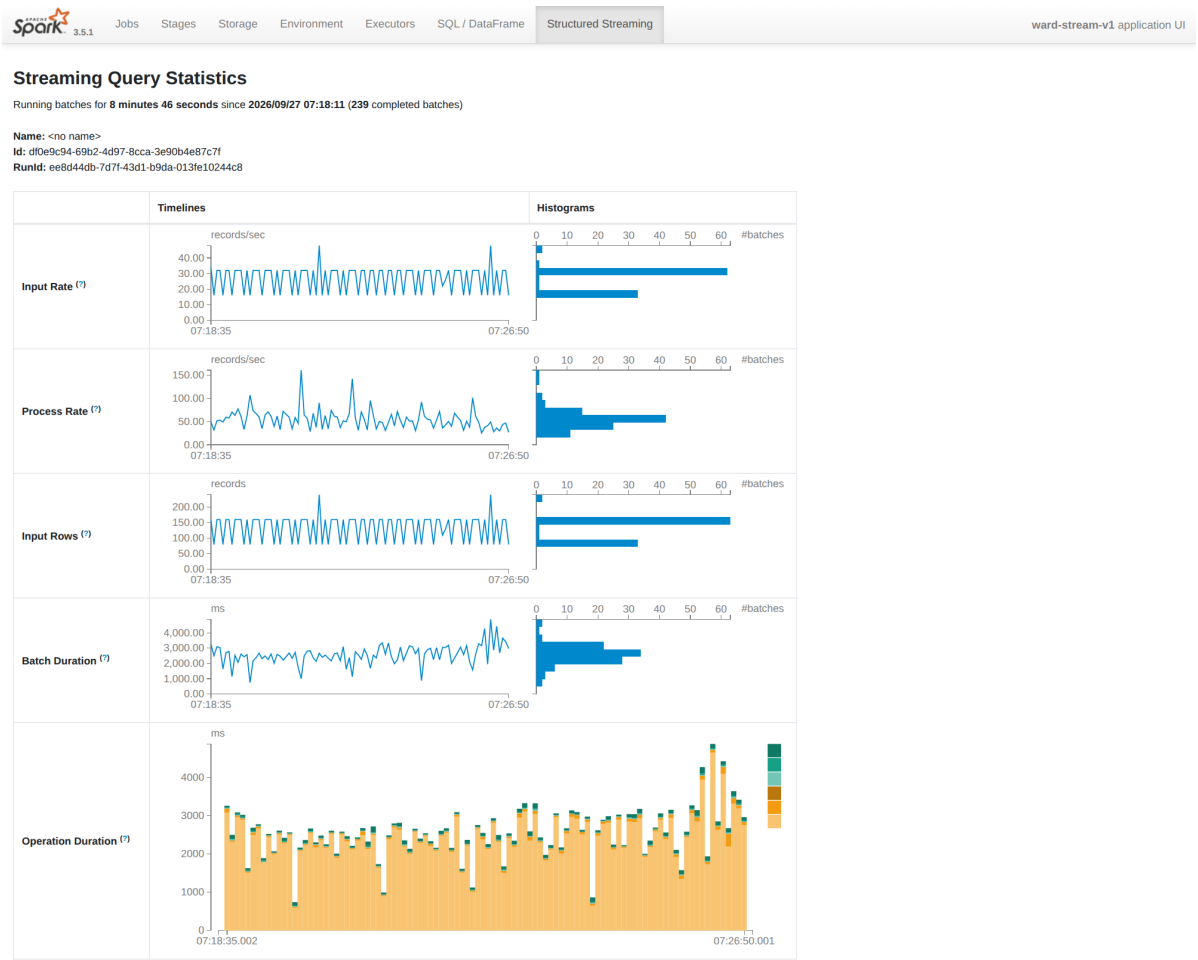


Figure 18: Spark UI: the live stream's input rate and batch duration.

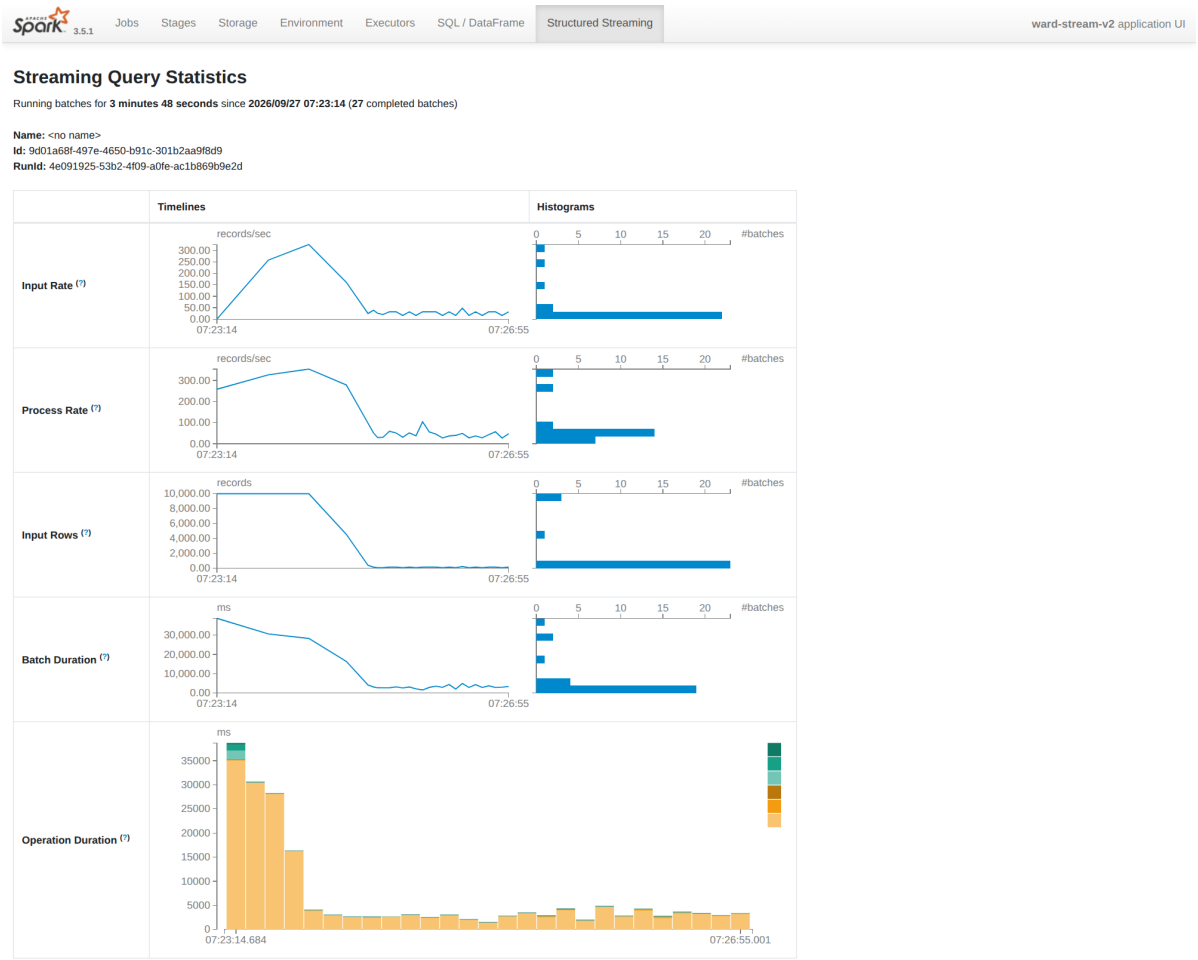


Figure 19: Spark UI: the replay stream catching up.

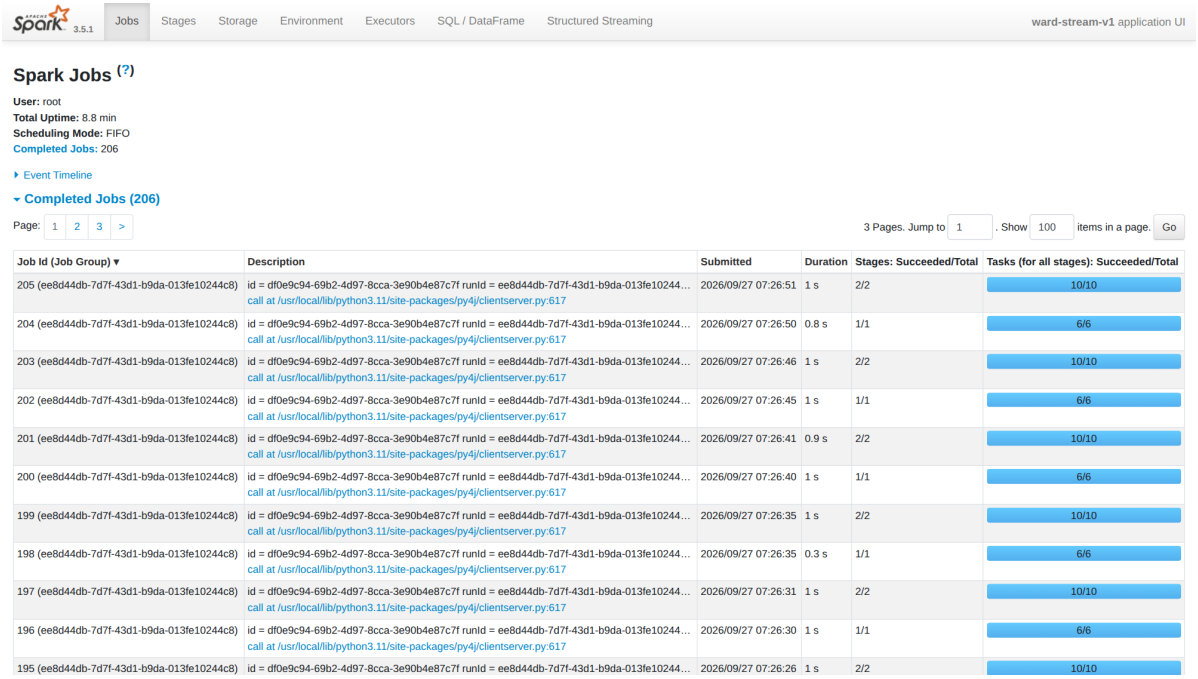


Figure 20: Spark UI: completed jobs.

```
cqlsh> SELECT patient_id, risk_score, news2_total, lab_contribution, risk_tier, scored_at FROM ward.ward_risk_snapshot WHERE ward_id='WARD-A' AND scorer_version='v1' LIMIT 12;

 patient_id | risk_score | news2_total | lab_contribution | risk_tier | scored_at
------------+------------+-------------+------------------+-----------+---------------------------------
       P014 |         14 |          10 |                4 |  CRITICAL | 2026-04-05 17:34:02.669000+0000
       P031 |          9 |           9 |                0 |      HIGH | 2026-04-05 11:21:48.180000+0000
       P001 |          8 |           8 |                0 |      HIGH | 2026-04-05 10:51:48.138000+0000
       P030 |          8 |           8 |                0 |      HIGH | 2026-04-05 09:51:48.148000+0000
       P031 |          8 |           8 |                0 |      HIGH | 2026-04-05 16:22:19.329000+0000
       P001 |          7 |           7 |                0 |      HIGH | 2026-04-05 11:06:48.229000+0000
       P002 |          7 |           7 |                0 |      HIGH | 2026-04-05 16:34:02.783000+0000
       P008 |          7 |           7 |                0 |      HIGH | 2026-04-05 15:07:19.315000+0000
       P031 |          7 |           7 |                0 |      HIGH | 2026-04-05 17:34:02.669000+0000
       P001 |          6 |           6 |                0 |    MEDIUM | 2026-04-05 09:51:48.148000+0000
       P002 |          6 |           6 |                0 |    MEDIUM | 2026-04-05 16:22:19.329000+0000
       P008 |          6 |           6 |                0 |    MEDIUM | 2026-04-05 17:34:02.669000+0000

(12 rows)
```

Figure 21: Cassandra: the ward snapshot partition, highest risk first.

```
cqlsh> SELECT scorer_version, scored_at, news2_total, clinical_risk FROM ward.risk_scores_by_patient WHERE patient_id='P031' AND scorer_version='v1' LIMIT 5; SELECT scorer_version, scored_at, news2_tota

 scorer_version | scored_at                       | news2_total | clinical_risk
----------------+---------------------------------+-------------+---------------
             v1 | 2026-04-05 17:34:02.669000+0000 |           7 |          HIGH
             v1 | 2026-04-05 17:19:02.751000+0000 |           5 |        MEDIUM
             v1 | 2026-04-05 17:04:02.739000+0000 |           7 |          HIGH
             v1 | 2026-04-05 16:49:02.745000+0000 |           7 |          HIGH
             v1 | 2026-04-05 16:34:02.783000+0000 |           7 |          HIGH

(5 rows)

 scorer_version | scored_at                       | news2_total | clinical_risk
----------------+---------------------------------+-------------+---------------
             v2 | 2026-04-05 17:34:02.669000+0000 |           4 |           LOW
             v2 | 2026-04-05 17:19:02.751000+0000 |           2 |           LOW
             v2 | 2026-04-05 17:04:02.739000+0000 |           4 |           LOW
             v2 | 2026-04-05 16:49:02.745000+0000 |           4 |           LOW
             v2 | 2026-04-05 16:34:02.783000+0000 |           4 |           LOW

(5 rows)
```

Figure 22: Cassandra: P031 scored under v1 and v2.

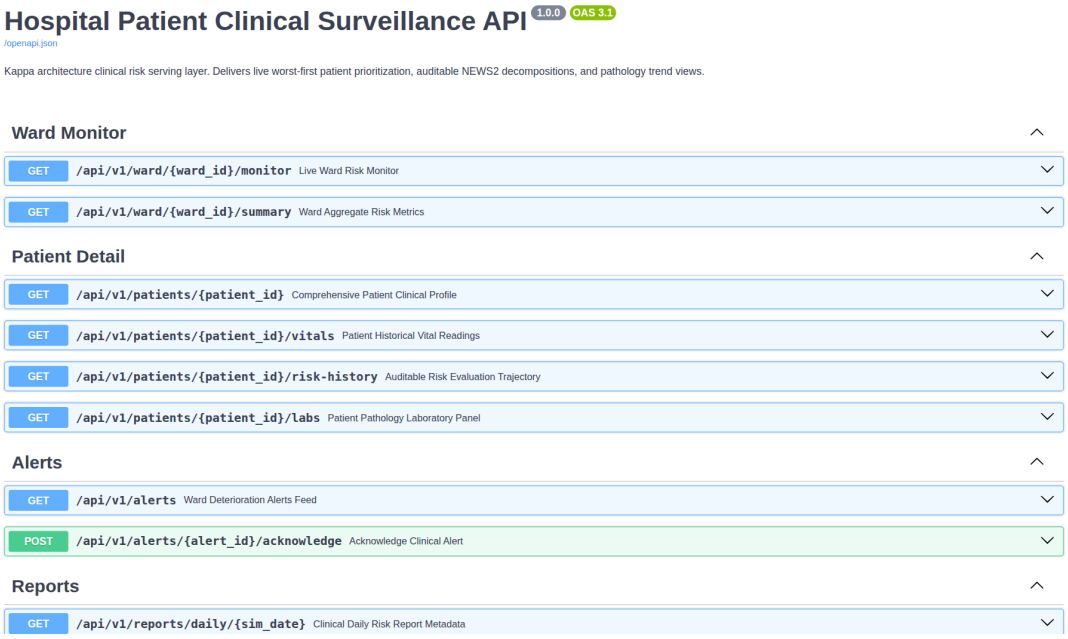


Figure 23: FastAPI documentation page.

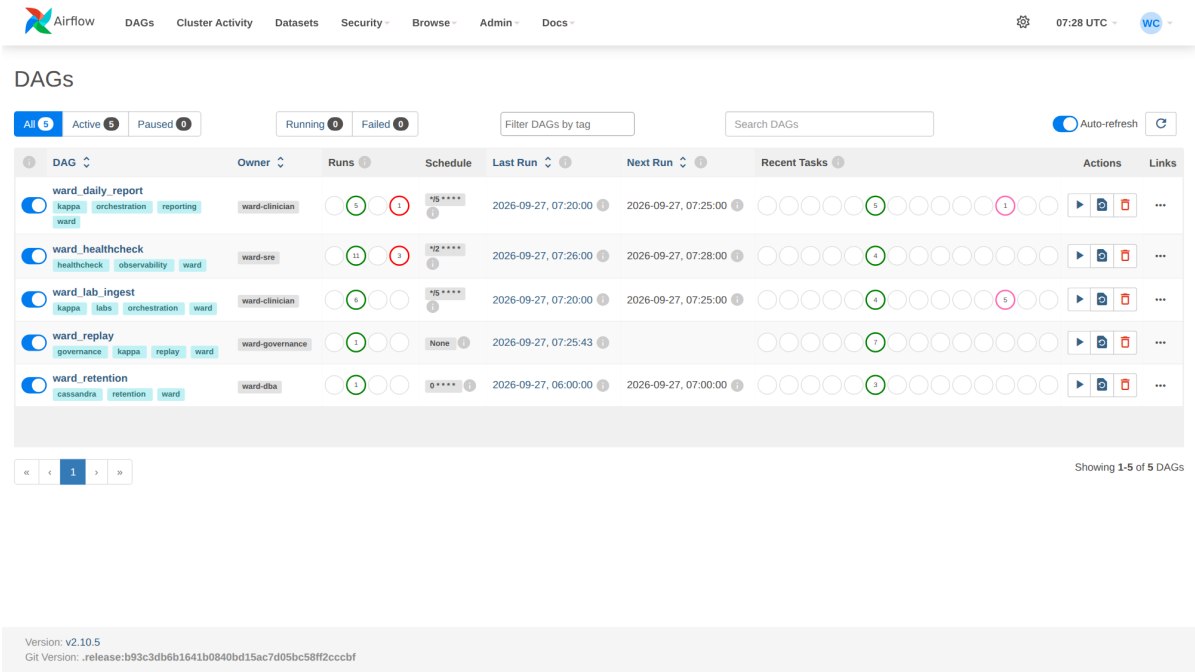


Figure 24: Airflow: the five workflows.

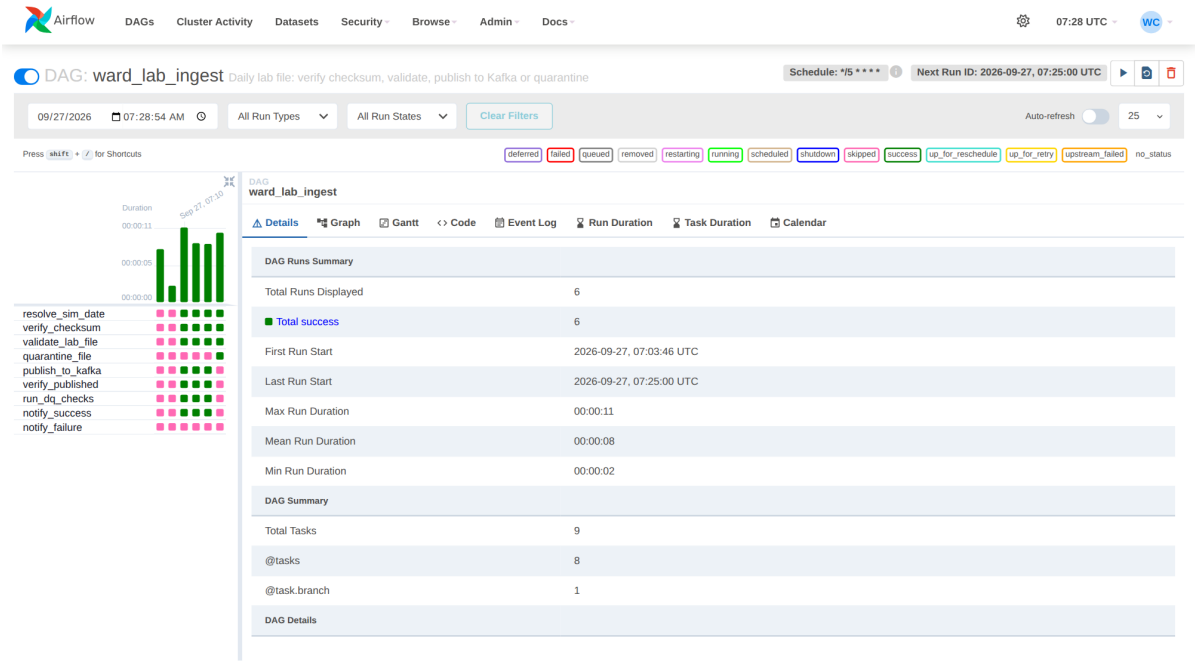


Figure 25: Airflow: `ward_lab_ingest`, one run per simulated day. The failed runs are the scripted bad days: the day-4 file was quarantined because of a corrupted reference range, and the day-5 file never arrived.

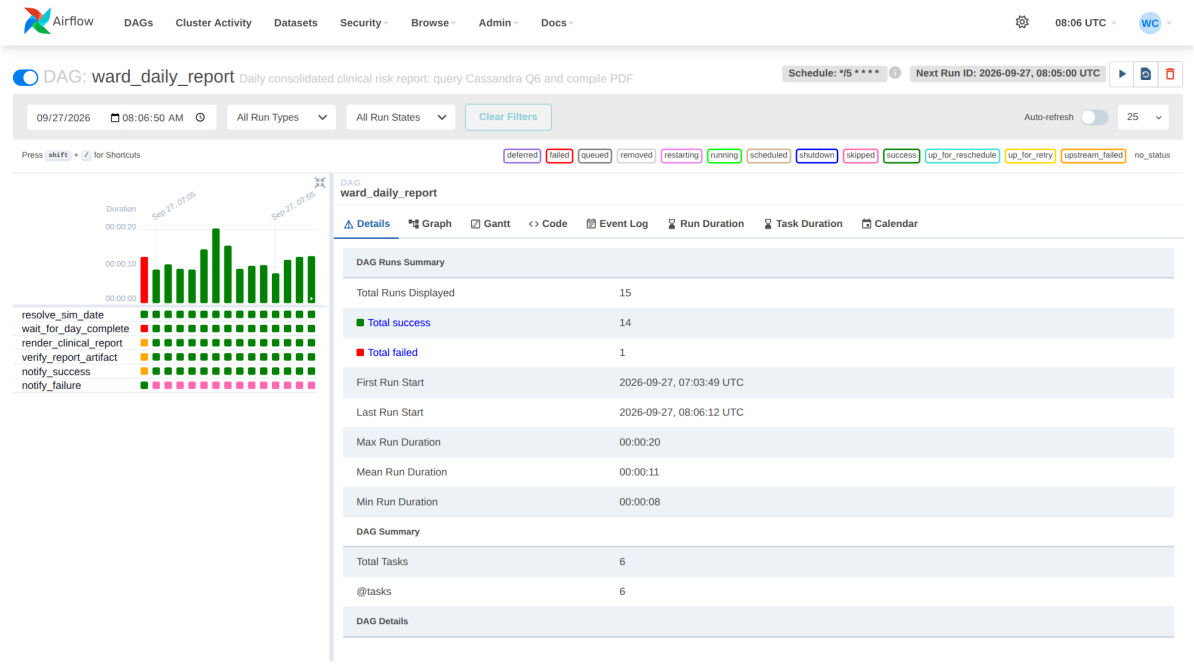


Figure 26: Airflow: `ward_daily_report` runs.

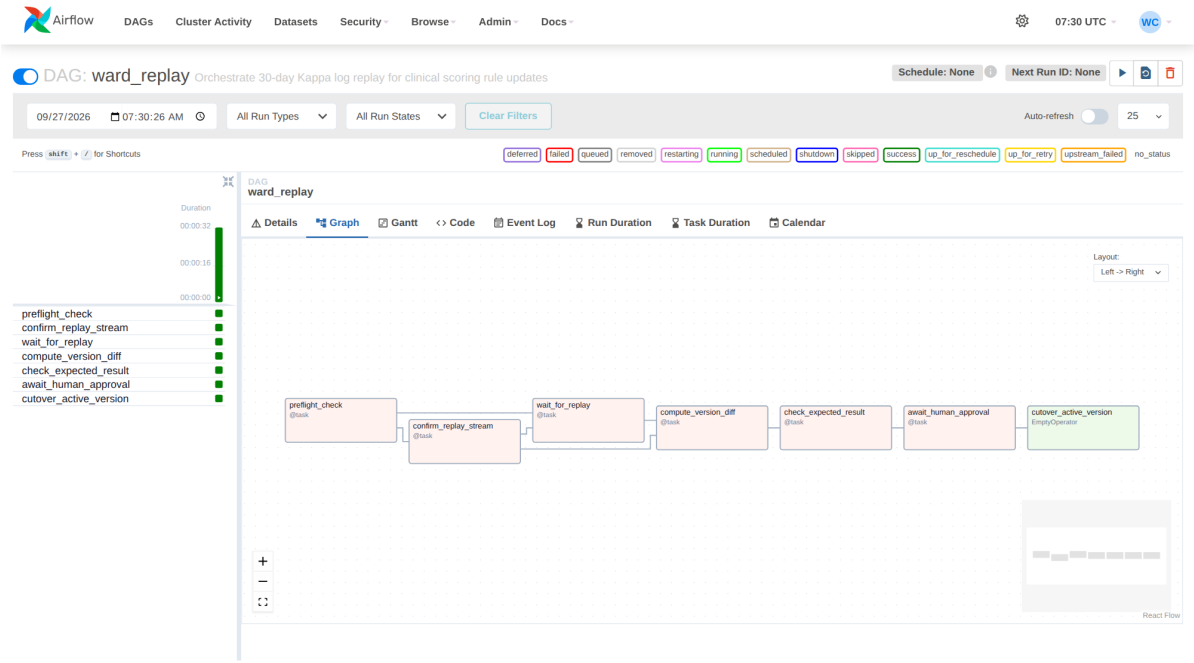


Figure 27: Airflow: `ward_replay` stops at the approval step.

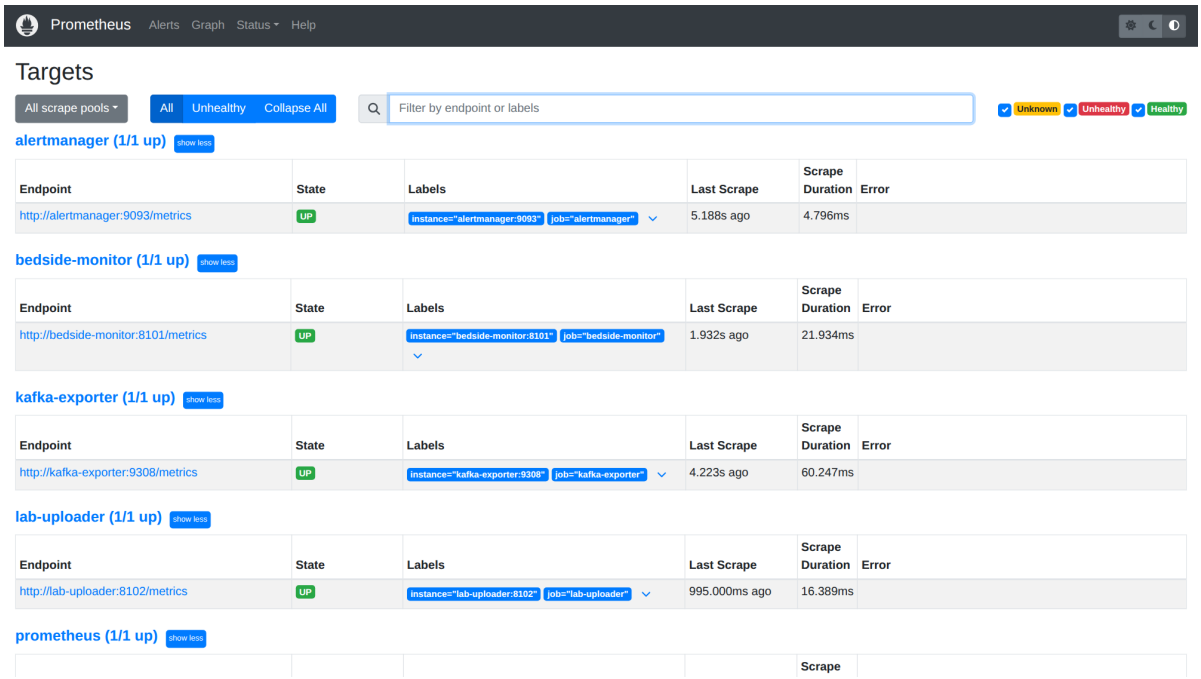


Figure 28: Prometheus: every scrape target up.

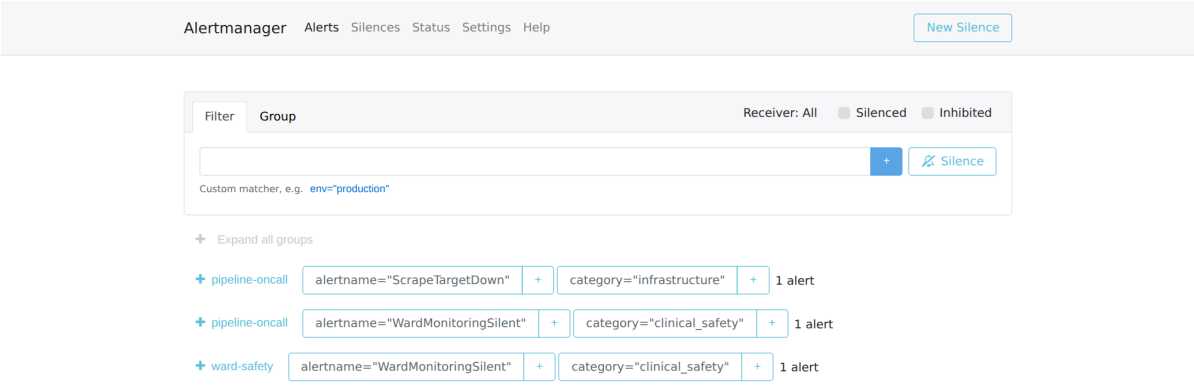


Figure 29: Alertmanager: `WardMonitoringSilent` routed to the ward and to the on-call engineer.

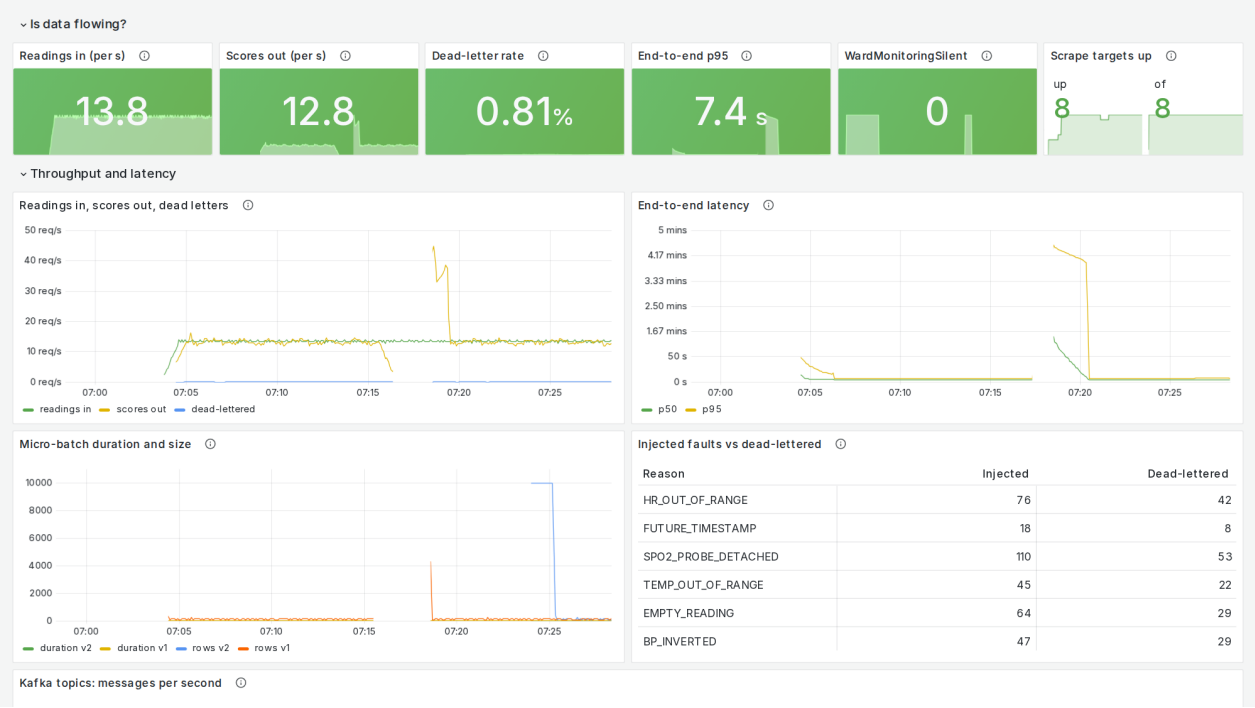


Figure 30: Grafana: pipeline health.

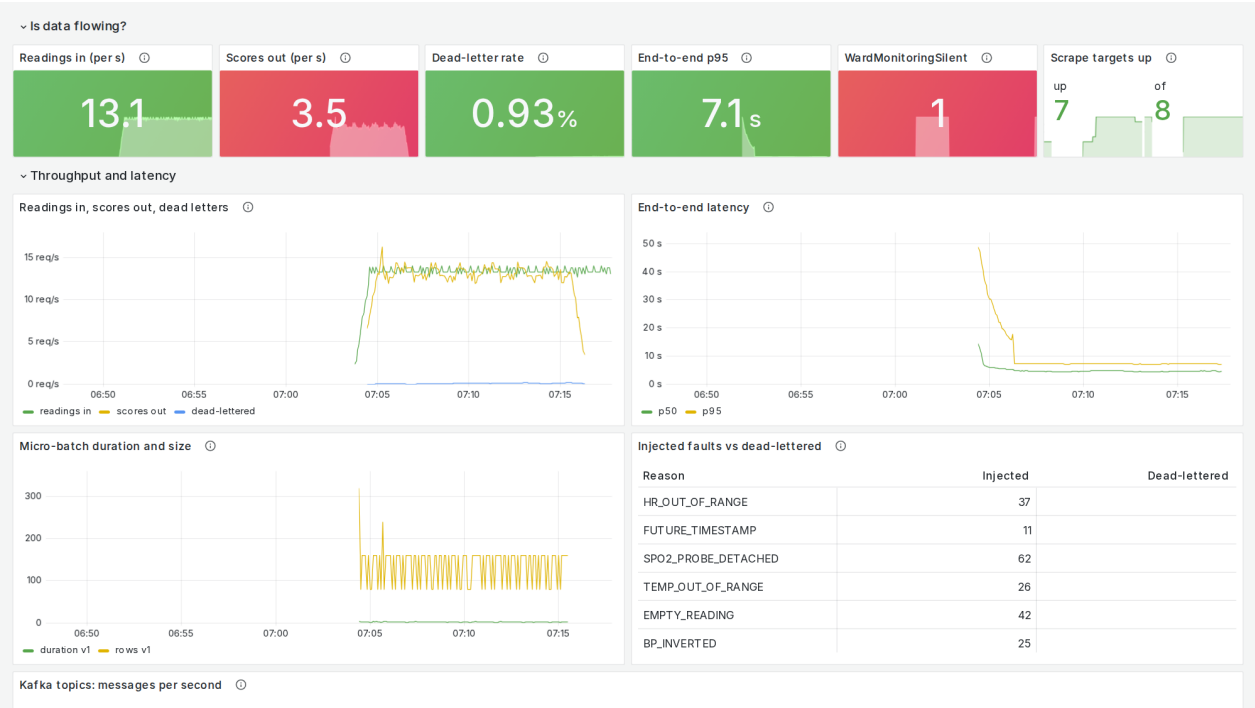


Figure 31: Grafana: pipeline health during the outage test.